14 haftalık yol haritasında 3. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 03

C’nin Yapı Taşları: Değişkenler, Veri Tipleri, Bellekte Temsil ve Temel Giriş/Çıkış

Bir değişken bellekte nasıl yaşar, sayılar nasıl bitlere dönüşür, klavyeden nasıl okuyup ekrana nasıl yazarız?

MODÜL II · VERİNİN TEMSİLİ VE İŞLENMESİ · H3–H4

HAFTANIN BÜYÜK SORUSU

Bilgisayar neden 0.1 + 0.2 işleminin sonucunu tam olarak 0.3 bulamaz ve bir sayaç neden birdenbire eksi değere düşer?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 3
1.1 Öğrenme çıktıları 3
2. Geçen Haftadan: Algoritmadan Koda 3
3. Kavramsal Anlatım 4
3.1 Bir C programının anatomisi 4
3.2 Tanımlayıcılar, anahtar sözcükler ve adlandırma 4
3.3 Değişken: adı, tipi, değeri ve adresi 4
3.4 Veri tipleri: kaç bayt, hangi aralık? 5
3.5 Tamsayılar bellekte: ikiye tümleyen ve taşma 6
3.6 Kayan noktalı sayılar ve IEEE 754 8
3.7 char ve ASCII: karakterler de birer sayıdır 9
3.8 Sabitler: değişmezler, #define ve const 10
3.9 printf ile biçimli çıktı 10
3.10 scanf ile giriş: adres, dönüş değeri ve tampon 11
4. Çözümlü Örnekler: Örnek Merdiveni 13
5. Hata Avcısı 24
6. Mühendis Gözüyle 25
6.1 Tip seçimi bir tasarım kararıdır 25
6.2 Birikimli hata: Patriot hesabını yeniden yapmak 26
7. Mini-OBS Projesi: v0.3 26
8. Alıştırmalar ve Atölye 27
8.1 Atölye etkinlikleri 27
8.2 Alıştırmalar 27
9. Kendini Değerlendirme 28
10. AP-2 Köprüsü ve Merak Kutusu 29
11. Özet, Kavram Haritası ve Terim Sözlüğü 29
11.1 Haftanın özeti 29
11.2 Büyük Sorunun yanıtı 29
11.3 Terim sözlüğü (Bölüm 6.8 terim standardıyla uyumlu) 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bilgisayar neden 0.1 + 0.2 işleminin sonucunu tam olarak 0.3 bulamaz ve bir sayaç neden birdenbire eksi değere düşer?

Bu hafta H2'deki "etiketli kutu"ların C karşılığıyla, **değişkenlerle** tanışıyoruz. Bir değişken yalnızca bir ad değildir: bellekte belirli sayıda bayt kaplar ve bu baytlardaki bitlerin yorumlanma kuralı, sayının hangi değerleri alabileceğini ve ne kadar hassas olabileceğini belirler. Haftanın sonunda iki sorunun yanıtını kendi makinenizde deneyerek vereceksiniz (özet yanıt: Bölüm 11.2).

> **GERÇEK DÜNYADAN · 25 Şubat 1991, Dhahran: 0,1 saniyenin bedeli**
>
> Körfez Savaşı sırasında Suudi Arabistan'ın Dhahran kentinde konuşlu bir Patriot füze savunma bataryası, yaklaşan bir Scud füzesini izleyip önleyemedi; füze bir ABD ordusu kışlasına isabet etti ve 28 Amerikan askeri hayatını kaybetti. ABD Genel Muhasebe Ofisi'nin (GAO) incelemesine göre batarya o gün 100 saati aşkın süredir kesintisiz çalışıyordu (GAO, 1992).
>
> Sistem zamanı onda bir saniyelik tıklarla bir tamsayı olarak sayıyor ve 0,1 ile çarparak saniyeye çeviriyordu. Ancak 0,1 ikili tabanda sonlu yazılamaz ve 24 bitlik yazmaçlara sığdırılırken kesilir. Her tıktaki küçük hata birikti: 100 saatin sonunda zaman yaklaşık **0,34 saniye** sapmıştı (raporda 0,3433 s); hedefin konumu yanlış hesaplandı ve füze izleme penceresinin dışında kaldı (GAO, 1992). Aynı hesabı Bölüm 6.2'de kendi programımızla yapacağız.

> **GERÇEK DÜNYADAN · 2014: Gangnam Style ve 32 bitlik sınır · 2038 problemi**
>
> Aralık 2014'te PSY'nin "Gangnam Style" videosu, görüntülenme sayısı 2 147 483 647'yi, yani 32 bitlik işaretli bir tamsayının alabileceği en büyük değeri ($2^{31} - 1$) aşan ilk YouTube videosu oldu; YouTube sayacı 64 bitlik bir tamsayıya yükseltti (Guinness World Records, t.y.).
>
> Aynı sınır, zamanı 1970'ten bu yana geçen saniye olarak 32 bitlik işaretli `time_t` ile tutan sistemleri 19 Ocak 2038 03:14:07 UTC'de bekliyor; bir sonraki saniyede pratikte tarih 13 Aralık 1901'e "düşer" (64 bitlik Linux'ta `time_t` 8 bayttır).

1.1 Öğrenme çıktıları

ÖÇ 3.1 Bir C programının yapısını (önişlemci yönergeleri, main fonksiyonu, deyimler, yorumlar) tanımlar. *(Bölüm 3.1)*

ÖÇ 3.2 Değişkeni ad, tip, değer ve adres nitelikleriyle açıklar; bildirim, ilk değer atama ve atama işlemlerini doğru kullanır. *(Bölüm 3.2–3.3)*

ÖÇ 3.3 Temel veri tiplerinin boyutlarını ve değer aralıklarını belirler; probleme uygun tipi gerekçelendirerek seçer. *(Bölüm 3.4, 6.1)*

ÖÇ 3.4 İkiye tümleyen gösterimi ve IEEE 754 kayan nokta gösterimini sezgisel düzeyde açıklar; taşma ve hassasiyet kaybını öngörür. *(Bölüm 3.5–3.6)*

ÖÇ 3.5 printf ve scanf ile biçimlendirilmiş giriş/çıkış yapar. *(Bölüm 3.9–3.10)*

2. Geçen Haftadan: Algoritmadan Koda

H2'de problemleri Girdi–İşlem–Çıktı (IPO) modeliyle çözümledik, sözde kod ve akış diyagramıyla ifade ettik, izleme tablosuyla elle yürüttük; değerleri "etiketli kutular"da tuttuk ve ← ile atadık. Bu hafta hepsinin C karşılığını göreceğiz: etiketli kutu bir değişkene (`int toplam;`), `toplam ← 0` ilk değer atamaya (`int toplam = 0`), `toplam ← toplam + i` atamaya (`toplam = toplam + i`) dönüşür; OKU ve YAZ paralelkenarları `scanf` ve `printf` çağrıları olur. C'de `=` eşitlik değil, atamadır.

TAHMİN ET · Geri çağırma: üç soru

- **S1.** "Çaya biraz şeker ekle." adımı algoritmanın beş özelliğinden (sonluluk, belirlilik, girdi, çıktı, etkinlik) hangisini ihlal eder? Nasıl düzeltirsiniz?
- **S2.** `toplam ← 0; i ← 1; i ≤ 3 olduğu sürece { toplam ← toplam + i; i ← i + 1 }` bittiğinde `toplam` ve `i` kaçtır? İzleme tablosuyla gösterin.
- **S3.** Akış diyagramında "kullanıcıdan bir sayı oku" adımı hangi sembolle çizilir? Bu adım C'de hangi fonksiyona dönüşecek?

Yanıtlar. S1: *belirlilik* ("1 çay kaşığı şeker ekle"). S2: (i, toplam) = (2, 1), (3, 3), (4, 6); `4 ≤ 3` yanlış olunca `toplam = 6`, `i = 4`. S3: paralelkenar; `scanf` ve `printf`. S2'de `toplam ← 0`'ı unutmak H2'nin Hata Avcısı'ydı; C'deki karşılığı bu haftanın HA3'üdür.

3. Kavramsal Anlatım

3.1 Bir C programının anatomisi

H1'deki ilk programımızın parçalarını adlarıyla tanıyalım; bu adlar derleyici iletilerini okurken ortak dilimiz olacak (Örnek 1'in listesini Bölüm 4.1'de izleyin).

- **Yorumlar** (`/* … */`, `// …`): derleyici yok sayar; neyi değil, *neden* yaptığınızı yazın. **Önişlemci yönergeleri** (preprocessor directives): `#` ile başlar, derlemeden önce işlenir; `#include <stdio.h>` `printf`/`scanf` bildirimlerini ekler; sonunda noktalı virgül yoktur.
- **main fonksiyonu**: `int main(void)` programın giriş noktasıdır; `return 0;` işletim sistemine "başarıyla bitti" der.
- **Bildirimler ve deyimler**: `int yas = 19;` bir bildirimdir; `yas = yas + 1;` ya da `printf(…);` noktalı virgülle biten deyimlerdir (statement). `{ }` deyimleri bir **blok** hâlinde toplar.

C büyük/küçük harfe duyarlıdır (`yas` ≠ `Yas`); boşluklar derleyici için önemsizdir, ama okunabilirlik için 4 boşluk girinti ve K&R stilini izliyoruz.

3.2 Tanımlayıcılar, anahtar sözcükler ve adlandırma

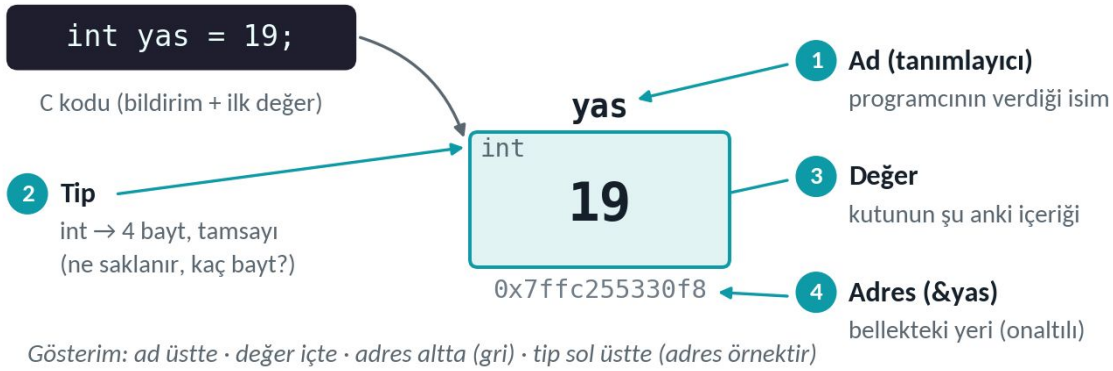
Değişkenlere, fonksiyonlara ve sabitlere verdiğimiz adlara **tanımlayıcı** (identifier) denir. Tanımlayıcı harf, rakam ve alt çizgi (_) içerebilir, rakamla başlayamaz, büyük/küçük harfe duyarlıdır ve bir **anahtar sözcük** (keyword) olamaz. C17'de 44 anahtar sözcük vardır (`int`, `char`, `double`, `const`, `sizeof`, `if`, `while`, `return` …). `_` ile başlayıp büyük harf ya da ikinci bir `_` ile devam eden adlar (`_Toplam`, `__x`) derleyici ve standart kütüphane için ayrılmıştır.

Örnekler: `ogrenci_sayisi` (geçerli, anlamlı: ders standardımız snake_case), `MAX_OGRENCI` (sabitler için BUYUK_HARF); `2vize` (rakamla başlıyor), `ortalama not` (boşluk), `final-notu` (`-` çıkarma operatörüdür) ve `int` (anahtar sözcük) geçersizdir. `öğrenci` gibi Türkçe harfli adlara izin derleyiciye bağlıdır; taşınabilirlik için `ogrenci` yazın. `x`, `a1` gibi anlamsız adlardan kaçının.

3.3 Değişken: adı, tipi, değeri ve adresi

Değişken (variable), programın çalışırken değer saklamak için kullandığı, adı olan bir bellek bölgesidir. Etiketli kutu benzetmesini bir adım ileri taşıyalım: kutunun bir **adı**, ne tür değer taşıyacağını ve kaç bayt olduğunu belirleyen bir **tipi**, o anki içeriği olan bir **değeri** ve bellekteki yerini gösteren bir **adresi** vardır (Şekil 3.1). Dönem boyunca kullanacağımız **bellek kutusu** gösterimi budur: ad üstte, değer içte, adres altta (onaltılı, gri), tip sol üst köşede. H12'de işaretçileri bu kutulara çizilen oklarla anlatacağız.

Şekil 3.1
Değişkenin dört niteliği ve ortak bellek kutusu gösterimi (adres örnektir)



Gösterim: ad üstte · değer içte · adres altta (gri) · tip sol üstte (adres örnektir)

C'de bir değişken kullanılmadan önce **bildirilir** (declaration): derleyiciye adını ve tipini söyleriz, derleyici tipe uygun büyüklükte yer ayırır. Bildirimle birlikte değer verilmesine **ilk değer atama** (initialization), sonradan değer yazılmasına **atama** (assignment) denir: `int yas = 19;` bildirim + ilk değer atama, `int ogrenci_sayisi;` yalnızca bildirim, `ogrenci_sayisi = 85;` atamadır. `=` işareti eşitlik değil, **atama operatörüdür**: "sağdaki ifadeyi oku ve hesapla, sonucu soldaki değişkene yaz". Bu yüzden `yas = yas + 1;` C'de çok doğaldır; eşitlik karşılaştırması `==` ile yapılır (H4–H5).

> KAPUTUN ALTINDA · **İlk değer atanmamış yerel değişken: "çöp" değer**
>
> `int ogrenci_sayisi;` satırı 4 baytlık yer ayırır ama o baytları temizlemez; içlerinde daha önce aynı bölgeyi kullanan kodun bıraktığı bitler durur. C standardı bu değeri **belirsiz** (indeterminate) sayar; okumak **tanımsız davranışa** (undefined behavior: standart hiçbir sonuç güvence altına almaz) yol açabilir.
>
> Hata Avcısı'ndaki HA3 programı `toplam = toplam + sayi;` satırıyla 10 yerine **32775** yazdırdı (Bölüm 5). Kural: **her değişkene bildirirken ilk değer verin.** (Fonksiyon dışında bildirilen değişkenler 0 ile başlar; H9.)

3.4 Veri tipleri: kaç bayt, hangi aralık?

Veri tipi (data type), bir değişkenin kaç bayt kaplayacağını ve bitlerin nasıl yorumlanacağını belirler. C'nin temel tipleri iki ailedir: **tamsayı tipleri** (`char`, `short`, `int`, `long`, `long long` ve `unsigned` hâlleri) ve **kayan noktalı tipler** (`float`, `double`).

Tablo 3.1
Temel veri tipleri, değişmezleri ve biçim belirteçleri

Tip	Ne saklar?	Değişmez	printf	scanf
`char`	Tek karakter ya da küçük tamsayı	`'A'`	`%c` (`%d`)	`%c`
`short`	Küçük tamsayı	`1200`	`%hd`	`%hd`
`int`	Tamsayı: varsayılan tercih	`42`	`%d`	`%d`
`unsigned int`	Negatif olmayan tamsayı	`42u`	`%u`	`%u`
`long`	Büyük tamsayı (boyutu platforma bağlı!)	`42L`	`%ld`	`%ld`
`long long`	Çok büyük tamsayı (en az 64 bit)	`42LL`	`%lld`	`%lld`
`float`	Tek duyarlıklı kesirli sayı	`3.14f`	`%f`	`%f`
`double`	Çift duyarlıklı kesirli: varsayılan	`3.14`	`%f` (`%lf`)	`%lf` (!)

Tamsayı tipleri varsayılan olarak **işaretlidir** (`int` = `signed int`); `unsigned` negatif değerlerden vazgeçip pozitif aralığı iki katına çıkarır. `char`'ın işaretliliği platforma bağlıdır: x86-64 Linux'ta işaretli, ARM tabanlı Linux'ta genellikle işaretsizdir. double yazdırırken printf'te `%lf` de kullanılabilir (C99'dan beri `%f` ile aynıdır); asıl tuzak scanf'tedir: double okumak için `%lf` **zorunludur**.

Bir tipin ya da değişkenin kaç bayt olduğunu `sizeof` operatörü verir; sonuç `size_t` tipindedir ve `%zu` ile yazdırılır. Aralıklar `<limits.h>` (`INT_MIN`, `INT_MAX`, `UINT_MAX`, `LLONG_MAX` …) ve `<float.h>` (`DBL_MAX`, `DBL_DIG` …) başlıklarındadır. Tablo 3.2, Örnek 6'nın bu ortamdaki (GCC 13, x86-64 Linux) **gerçek çıktısıdır**; son sütun Windows'taki farkı gösterir.

Tablo 3.2
Örnek 6'nın gerçek çıktısı: tip boyutları ve aralıkları (Linux x86-64, LP64)

Tip	Bayt	En küçük değer	En büyük değer	Windows x64 (LLP64) bayt
`char`	1	-128	127	1
`short`	2	-32768	32767	2
`int`	4	-2147483648	2147483647	4
`unsigned int`	4	0	4294967295	4
`long`	8	-9223372036854775808	9223372036854775807	**4** (±2 147 483 647)
`long long`	8	-9223372036854775808	9223372036854775807	8
`float`	4	−3.402823e+38	3.402823e+38 (≈ 6 basamak)	4
`double`	8	−1.797693e+308	1.797693e+308 (≈ 15 basamak)	8

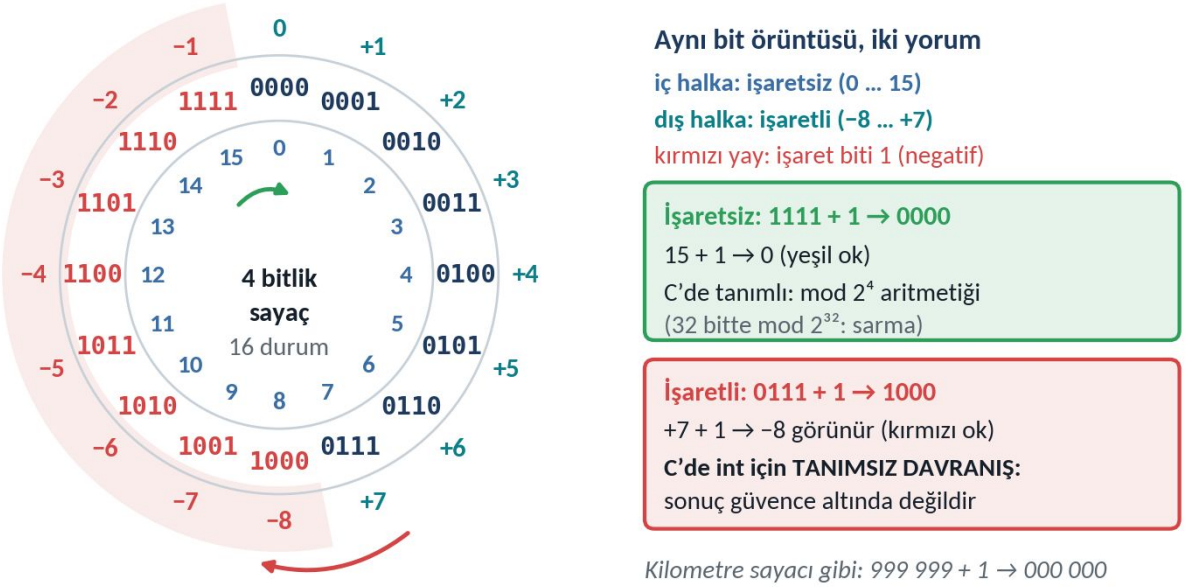
> KAPUTUN ALTINDA · **LP64 ve LLP64: long neden bazen 8, bazen 4 bayt?**
>
> - C standardı (ISO/IEC, 2018) yalnızca **en küçük** aralıkları belirler: `int` en az 16 bit, `long` en az 32 bit, `long long` en az 64 bit ve `char` ≤ `short` ≤ `int` ≤ `long` ≤ `long long`. Gerisini platform seçer.
> - 64 bitlik Linux ve macOS **LP64** modelini (long ve işaretçi 64 bit), 64 bitlik Windows **LLP64** modelini (long 32 bit; long long ve işaretçi 64 bit) kullanır. Örnek 6'yı Windows'ta MinGW-w64 ile derlerseniz long satırında 4 bayt görürsünüz. Aralık önemliyse `long long` kullanın; boyutları varsaymayın, `sizeof` ile ölçün (Atölye 3).

3.5 Tamsayılar bellekte: ikiye tümleyen ve taşma

8 bitlik işaretsiz bir tamsayıda bitlerin ağırlıkları 128, 64, …, 1'dir: `0000 0101` = 5, `1111 1111` = 255. Negatif sayılar, yaygın işlemcilerin tamamında **ikiye tümleyen** (two's complement) gösterimle saklanır (Bryant & O'Hallaron, 2016); C23 standardı bu gösterimi zorunlu kılar. En soldaki bitin ağırlığı negatiftir (8 bitte −128); bir sayının negatifi, bitleri ters çevirip 1 ekleyerek bulunur: +5 = `0000 0101` → ters çevir `1111 1010` → 1 ekle `1111 1011` = −5 (sağlama: −128 + 64 + 32 + 16 + 8 + 2 + 1 = −5). Bu sayede toplama devresi negatif sayılar için de değişmeden çalışır.

Sabit sayıda bit, sonlu sayıda durum demektir: 32 bit 2^{32} = 4 294 967 296 durum. Durumları bir çembere dizersek (Şekil 3.2) **taşma** (overflow) görünür hâle gelir: en büyük değere 1 eklenince çemberin başına dönülür, tıpkı 999 999'dan 000 000'a dönen bir kilometre sayacı gibi. İç halka bitlerin işaretsiz, dış halka işaretli yorumudur.

Şekil 3.2
4 bitlik sayı çemberi: iç halka işaretsiz (0…15), dış halka işaretli (−8…+7) yorum



HATA AVCISI · **C standardı ne diyor? Tanımlı sarma, tanımsız taşma**

- **İşaretsiz tamsayılar taşmaz, sarar:** `unsigned` işlemler 2 üzeri n modunda tanımlıdır; `UINT_MAX + 1` her makinede 0'dır.
- **İşaretli tamsayı taşması tanımsız davranıştır:** `INT_MAX + 1` için standart bir sonuç garanti etmez. Çoğu makinede donanım bitleri çember gibi döndürdüğü için −2147483648 görürsünüz; ama derleyici taşmanın hiç olmayacağını varsayarak kodu dönüştürebilir, bu "sarmaya" güvenilemez (Örnek 11; Merak Kutusu, Bölüm 10).
- Değer derlemede bilinirse GCC uyarır: `int x = INT_MAX + 1;` için "integer overflow in expression of type 'int' results in '-2147483648' [-Woverflow]". Örnek 11 bu yüzden eklenecek değeri kullanıcıdan okur.

Kaputun altında: bayt sırası ve küçük-uçluluk

4 baytlık bir `int` ardışık dört adrese yerleşir. x86-64 işlemciler **küçük-uçludur** (little-endian): en anlamsız bayt en küçük adrese yazılır; 0x12345678 bellekte `78 56 34 12` olarak durur (Şekil 3.3). Örnek 9'da gdb ile okuduğumuz gerçek baytlar da bunu doğrular: 42 değerindeki `x` için `2a 00 00 00`. Baytlara C kodu içinden tek tek erişmek işaretçi gerektirir; bunu **H12'de kodla göreceğiz**.

Şekil 3.3
0x12345678 değerinin küçük-uçlu ve büyük-uçlu bayt dizilişi



Aynı bitler, farklı yorum

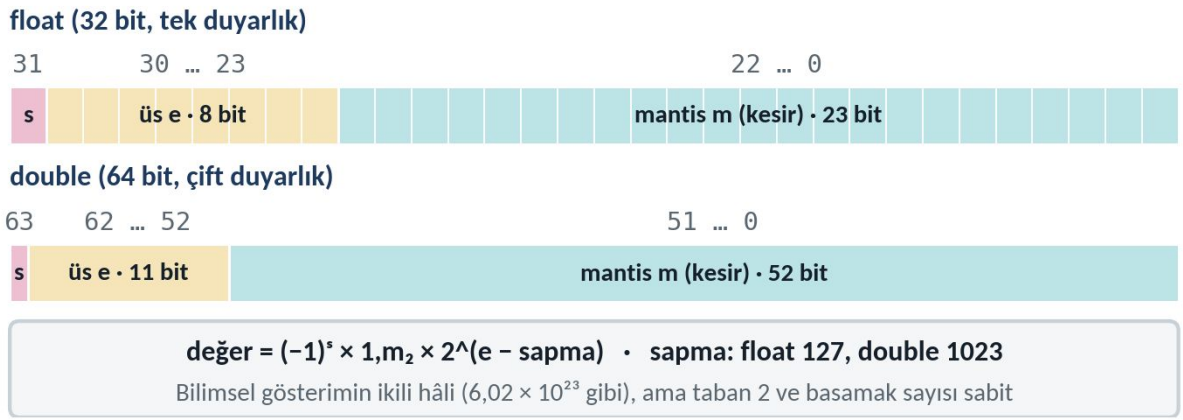
Bellekteki bitler kendi başına anlam taşımaz; anlamı **tip** verir. Ek örnek 14'te −1, `unsigned int u = i;` ile işaretsiz bir değişkene aktarılır: 32 bitin hepsi 1'dir (`ffffffff`) ve işaretsiz okununca 4294967295 eder; 8 bitlik `1100 1000` deseni `signed char` olarak −56, `unsigned char` olarak 200'dür. İşaretliyi işaretsize aktarmak tanımlıdır (2 üzeri n modunda); sığmayan bir değeri işaretli tipe aktarmanın sonucu ise derleyiciye bağlıdır (implementation-defined; H4).

```
ÇIKTI
$ ./bitler
int olarak       : -1
unsigned olarak  : 4294967295
onaltılı (%x)    : ffffffff
signed char      : -56
unsigned char    : 200
```

3.6 Kayan noktalı sayılar ve IEEE 754

Kesirli sayılar için `float` (4 bayt) ve `double` (8 bayt) vardır; ikisi de IEEE 754 ikili kayan nokta biçimini kullanır (Goldberg, 1991; IEEE, 2019). Sayı, bilimsel gösterimdeki gibi bir **işaret**, bir **üs** ve bir **mantis** (kesir) olarak saklanır (Şekil 3.4): değer = $(-1)^s \times 1{,}m_2 \times 2^{(e - sapma)}$; sapma (bias) float'ta 127, double'da 1023'tür. Mantisin baştaki 1'i saklanmaz (gizli bit); double 53 bitlik hassasiyetle yaklaşık 15–17 anlamlı onluk basamak (`DBL_DIG` = 15), float yaklaşık 6–9 basamak (`FLT_DIG` = 6) sağlar.

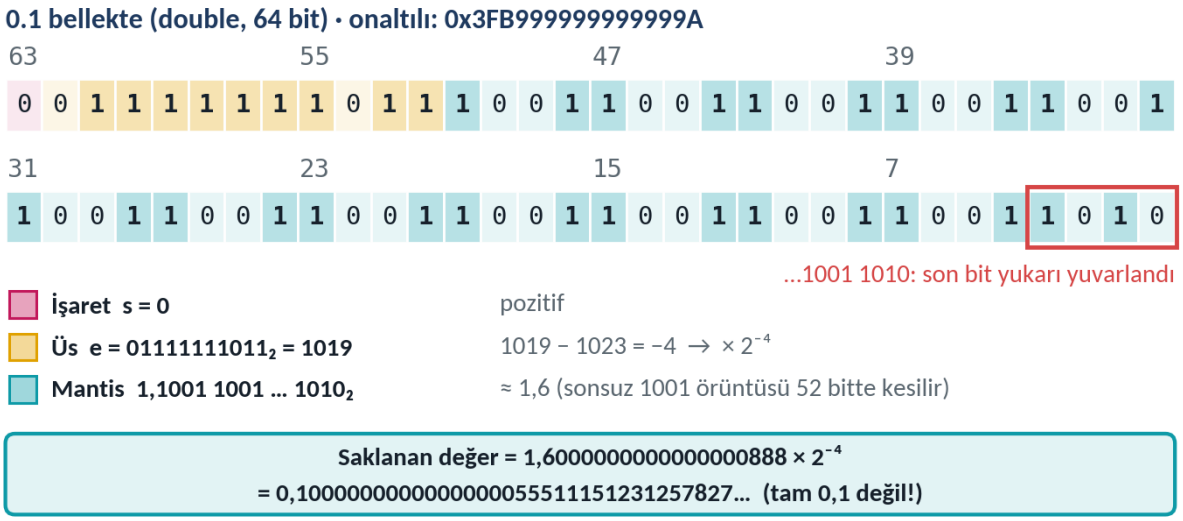
Şekil 3.4
IEEE 754 bit alanları: float (1 | 8 | 23) ve double (1 | 11 | 52)



0.1 neden tam saklanamaz?

Onluk tabanda 1/3 = 0,333… sonlu yazılamaz; ikili tabanda da paydası 2'nin kuvveti olmayan kesirler sonlu yazılamaz. 0,1'i ikiliye çevirmek için kesirli kısım art arda 2 ile çarpılıp tam kısımlar yazılır: 0,2 → 0; 0,4 → 0; 0,8 → 0; 1,6 → 1; 1,2 → 1; ve yeniden 0,2'ye döneriz. Sonuç $0{,}0\,0011\,0011\,0011\ldots_2$'dir: `0011` sonsuza dek tekrar eder. double bu açılımın ilk 53 anlamlı bitini alır ve sonrasına bakarak **yuvarlar** (Şekil 3.5).

Şekil 3.5
0.1'in bellekteki 64 biti (0x3FB999999999999A): işaret, üs ve mantis



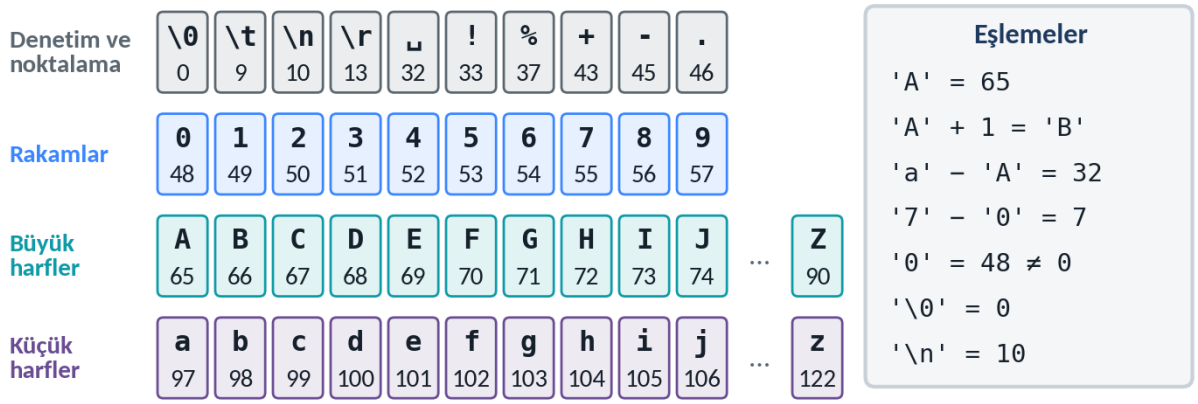
Saklanan değer 0,1000000000000000055511151231257827… olur; `0.2` ve `0.3` de en yakın double'a yuvarlanır. 0.1 + 0.2 toplamı da yuvarlanır ve `0.3` yazınca saklanan double'ın (0,29999999999999998890) **bir sonraki komşusuna** (0,30000000000000004441) düşer; aradaki fark tam bir adımdır: $2^{-54} \approx 5{,}55 \times 10^{-17}$. Örnek 12'de `toplam - 0.3` tam olarak bu kadar çıkar (5.551115e-17). Paydası 2'nin kuvveti olan kesirler ise tam saklanır: 0.5 + 0.25, `%.20f` ile `0.75000000000000000000` yazar.

> HATA AVCISI · **Kayan noktalı sayıları == ile karşılaştırmayın**
>
> Örnek 12'de `toplam == 0.3` ifadesi 0 (yanlış) verir. Doğru yaklaşım, farkın küçük bir eşikten (tolerans, ε) küçük olup olmadığına bakmaktır: $|a - b| < 10^{-9}$; `fabs` ve karşılaştırma operatörleri H4–H5'te gelecek. Parayı kuruş cinsinden tamsayı tutmak yaygın bir tercihtir. Varsayılan kesirli tip `double`'dır; `0.1f` float olarak 0,100000001490116… diye saklanır.

3.7 char ve ASCII: karakterler de birer sayıdır

`char` 1 baytlık bir tamsayı tipidir. Ekranda karakter olarak gördüğümüz şey bellekte bir sayıdır: ASCII tablosunda `'A'` 65, `'a'` 97, `'0'` 48'dir (Şekil 3.6). Bu yüzden karakterlerle aritmetik yapılabilir: `'A' + 1` → `'B'`, `'7' - '0'` → 7, küçük harften büyüğe geçmek için 32 (`'a' - 'A'`) çıkarılır (Örnek 7).

Şekil 3.6
ASCII tablosundan bir kesit ve char ↔ int eşlemesi



char 1 baytlık bir tamsayıdır: ekranda karakter, bellekte sayı (ASCII kodu). ␣ = boşluk; ' ' karakter, " " dizi içindir.

- Karakter değişmezi **tek tırnakla** yazılır (`'A'`); çift tırnak (`"A"`) bir karakter dizisidir (H13). `'7'` (55) ile `7` farklıdır.
- Kaçış dizileri de tek bir karakterdir: `'\n'` (10), `'\t'` (9), `'\0'` (0). `'A'` değişmezinin tipi aslında `int`'tir: bu ortamda `sizeof('A')` 4 verir.
- ASCII 7 bitliktir ve Türkçe harfleri içermez. `'ş'` UTF-8'de 2 bayttır; `char c = 'ş';` için GCC "multi-character character constant" uyarısı verir (H13).

3.8 Sabitler: değişmezler, #define ve const

Koda doğrudan yazılan değerlere **değişmez** (literal) denir; yazılış tipi belirler: `42` int, `42u` unsigned int, `42L` long, `42LL` long long, `0x2A` onaltılı (= 42), `052` **sekizli** (= 42; baştaki 0'a dikkat!), `3.14` double, `3.14f` float, `1.5e-3` = 1,5 × 10^{-3} (double), `'A'` karakter (değeri 65), `"Merhaba"` karakter dizisi. Anlamı olan sabitleri (π, dönüşüm katsayısı) çıplak sayılar yerine adlandırırız; iki yolu vardır. Kod standardımızda sabit adları BUYUK_HARF'tir; Örnek 4 ikisini birlikte kullanır.

`#define PI 3.14159265358979` satırı önişlemci tarafından derlemeden önce metin olarak yerine konur: tipi ve adresi yoktur, sonuna noktalı virgül konmaz (konursa metne dahil olur!). `const double PI = 3.14159265358979;` ise derleyicinin gördüğü tipli, salt okunur bir değişkendir: adresi vardır, hata ayıklayıcıda görünür ve değiştirilmeye çalışılırsa GCC "assignment of read-only variable" hatası verir.

3.9 printf ile biçimli çıktı

`printf`'in ilk argümanı **biçim dizisidir** (format string); `%` ile başlayan her **biçim belirteci** (format specifier) sonraki argümanlardan birinin nasıl yazılacağını söyler: `%[bayrak][genişlik][.hassasiyet]dönüşüm` (Kernighan & Ritchie, 1988). Örneğin `%-8.2f`: `-` sola yasla, `8` alan genişliği, `.2` iki ondalık basamak, `f` double. Belirteç argümanın tipiyle **uyuşmalıdır**; uyuşmazlık tanımsız davranıştır (Bölüm 5, HA2). Tablo 3.3'teki çıktıların hepsi bu ortamda gerçek bir C programıyla alınmıştır; köşeli parantezler alan genişliğini görünür kılar. `%p` adresi yazar (Örnek 9), `%%` yüzde işaretinin kendisidir.

Tablo 3.3
printf biçim belirteçleri ve gerçek çıktıları

Belirteç	Argüman tipi	Örnek değer	Gerçek çıktı
`%d`, `%5d`, `%-5d`, `%05d`, `%+d`	int	`42`	`[42]` `[   42]` `[42   ]` `[00042]` `[+42]`
`%u`	unsigned int	`4294967295u`	`[4294967295]`
`%ld`, `%lld`	long, long long	`9223372036854775807L`, `-5LL`	`[9223372036854775807]` `[-5]`
`%zu`	size_t (sizeof)	`sizeof(int)`	`[4]`
`%f`, `%.2f`	double	`3.14159`	`[3.141590]` `[3.14]`
`%8.2f`, `%-8.2f`, `%08.2f`	double	`3.14159`	`[    3.14]` `[3.14    ]` `[00003.14]`
`%e`, `%.3e`, `%g`	double	`1234.5678`	`[1.234568e+03]` `[1.235e+03]` `[1234.57]`
`%c`	int (char)	`'A'`	`[A]`
`%x`, `%X`, `%o`, `%#x`	unsigned int	`255`	`[ff]` `[FF]` `[377]` `[0xff]`

> KAPUTUN ALTINDA · **printf yuvarlarken ne yapar?**
>
> `%.2f` yazarken yuvarlar ama bellekteki değeri değiştirmez. glibc saklanan ikili değerin tam açılımına bakar, tam ortadakileri çift rakama yuvarlar: `printf("%.0f", 2.5)` → `2`, `printf("%.0f", 3.5)` → `4`, `printf("%.2f", 0.125)` → `0.12`. `2.675` bellekte 2,67499999… olarak saklandığı için `%.2f` ile `2.67` çıkar.

3.10 scanf ile giriş: adres, dönüş değeri ve tampon

`scanf` standart girişten okuduğu karakterleri biçim dizisine göre çevirip değişkenlere yazar. `printf`'ten farklı olarak scanf'e değişkenin **değerini değil adresini** veririz (King, 2008): `scanf("%d", &a);`. **&** (adres) operatörü "a'nın bellekteki yeri" demektir; scanf adresi bilmezse yazamaz. Bu fikri H12'de işaretçilerle derinleştireceğiz; şimdilik kural: **scanf'te her değişkenin önüne & koyun** (karakter dizileri hariç, H13). Belirteçler: int `%d`, unsigned `%u`, long long `%lld`, float `%f`, double `%lf` (printf'te `%f` yeterlidir, `%lf` de geçerlidir; scanf'te `%f` float içindir!), char `%c`.

scanf'in dönüş değeri

scanf **başarıyla okuyup atadığı öge sayısını** döndürür; ilk çevrimden önce dosya sonuna gelinirse `EOF` (bu sistemde −1) döndürür. Okuma başarısız olursa değişkene dokunulmaz; ilk değer vermemizin bir nedeni budur. Ek örnek 13'te `int okunan = scanf("%d %d", &a, &b);` satırının dört gerçek çalıştırması Tablo 3.4'tedir. H6'dan itibaren her okumada dönüş değerini denetleyeceğiz.

Tablo 3.4
Ek örnek 13'ün gerçek çıktıları (a ve b 0 ile başlatıldı)

Girdi	Dönüş değeri	a	b
`12 30`	2	12	30
`12 abc`	1	12	0
`abc`	0	0	0
Ctrl+D (dosya sonu)	-1	0	0

Giriş tamponu ve karakter okuma tuzağı

Klavyeden yazdıklarınız Enter'a basılınca, sondaki `'\n'` dahil, **giriş tamponuna** (input buffer) düşer; scanf buradan okur ve **okumadığını bırakır** (Şekil 3.7). `%d` sayıyı okuyup `'\n'`i bırakır; ardından gelen `%c` bu `'\n'`i hemen alır ve kullanıcıya sormadan geçer. Çözüm `%c`'nin önüne bir boşluk koymaktır: biçim dizisindeki boşluk "beyaz karakterlerin (boşluk, sekme, yeni satır) hepsini atla" demektir (`%d`, `%lf` bunları zaten atlar). Ek örnek 15'in gerçek çalıştırmasında birinci deneme isteminin hemen ikinciye geçtiğine dikkat edin.

Şekil 3.7
scanf'in giriş tamponundan okuması ve " %c" çözümü



H03_15_karakter_tampon.c · 15–19. satırlar

```
    scanf("%d", &yas);            /* 19 okunur, '\n' tamponda kalır */
    printf("Bir harf (1. deneme): ");
    scanf("%c", &harf1);          /* bekleyen '\n' hemen okunur! */
    printf("Bir harf (2. deneme): ");
    scanf(" %c", &harf2);         /* boşluk: beyaz karakterleri atla */
```

ÇIKTI

```
$ ./tampon
Yaşınız: 19
Bir harf (1. deneme): Bir harf (2. deneme): B

yas = 19
harf1 kodu = 10 (10, '\n' karakterinin kodudur)
harf2 = 'B'
```

4. Çözümlü Örnekler: Örnek Merdiveni

Her basamak bir öncekine yalnızca bir yeni öge ekler (Tablo 3.5). Önce kodu okuyup çıktıyı **tahmin edin**, sonra çalıştırıp karşılaştırın (PRIMM). Çıktılar bu ortamda gerçekten derlenip çalıştırılarak alınmıştır (`kodlar/ciktilar/`); etkileşimli programlarda kullanıcının yazdığı değer istemin sonundadır. Yer kazanmak için bazı örneklerde yalnızca yeni satırlar (kesit) gösterilir; **amiral örnekler** (5 ve 10) ile Örnek 11 ise başlık yorumu dışında tam olarak, dosyadaki satır numaralarıyla verilir.

Tablo 3.5
Haftanın örnek merdiveni (dosyalar: kodlar/H03_01_… – H03_12_…)

#	Örnek	Yeni öge / amaç	Düzey
1	Değişken bildirip değerini yazdırma	Bildirim, ilk değer, printf %d	★
2	Klavyeden iki sayı okuyup toplamını yazdırma	scanf ve &	★
3	Dikdörtgenin alanı ve çevresi	Formülün koda aktarılması	★
4	Dairenin alanı	double, const/#define PI, %.2f	★
5	Celsius → Fahrenheit dönüşümü	9/5 tamsayı bölmesi tuzağına ilk temas	★★
6	Makinenizde veri tipi boyutları tablosu	sizeof, limits.h; platform farkı	★★
7	Karakter ↔ ASCII kodu dönüşümü ('A' + 1)	char'ın sayısal doğası	★★
8	İki değişkenin değerlerini takas etme	Geçici değişken (üçüncü bardak analojisi)	★★
9	Değişkenlerin adreslerini yazdırma	&, %p; bellek hücresi modeli	★★
10	Vücut kitle indeksi hesaplayıcı	Bütünleştirici: giriş, işlem, biçimli çıktı	★★
11	Tamsayı taşması deneyi (INT_MAX + 1)	İkiye tümleyen; taşma	★★★
12	0.1 + 0.2 deneyi ve %.20f	Kayan nokta hassasiyeti	★★★

4.1 Örnek 1 ★ Değişken bildirip değerini yazdırma

Problem. Bir öğrencinin yaşını ve sınıftaki öğrenci sayısını değişkenlerde tutup yazdıralım; bir yıl sonraki yaşı hesaplayalım.

H03_01_degisken_yazdir.c · 10–18. satırlar

```c
    int yas = 19;              /* bildirim + ilk değer atama */
    int ogrenci_sayisi;        /* yalnızca bildirim */

    ogrenci_sayisi = 85;       /* atama: kutuya değer yazılır */
    printf("Yaş: %d\n", yas);
    printf("Sınıftaki öğrenci sayısı: %d\n", ogrenci_sayisi);

    yas = yas + 1;             /* sağ taraf okunur, sonuç sola yazılır */
    printf("Bir yıl sonra yaş: %d\n", yas);
```

ÇIKTI

```
$ ./degisken
Yaş: 19
Sınıftaki öğrenci sayısı: 85
Bir yıl sonra yaş: 20
```

Yeni ögeler: bildirim + ilk değer (`int yas = 19;`), yalnızca bildirim ve ardından atama, `%d` ile yazdırma; argümanlar belirteçlerle sırayla eşleşir. **Değiştir:** (a) `int yas = 19;` satırındaki `= 19`'u silip derleyin; `-Wall` ne der? (b) `int dogum_yili = 2007;` ekleyip yaşı `2026 - dogum_yili` ile hesaplayın.

4.2 Örnek 2 ★ Klavyeden iki sayı okuyup toplamını yazdırma

Problem. Kullanıcıdan iki tamsayı alıp toplamlarını yazdıralım.

H03_02_iki_sayi_toplam.c · 10–19. satırlar

```c
    int a;
    int b;

    printf("Birinci sayı: ");
    scanf("%d", &a);           /* &a: "a'nın adresine yaz" */
    printf("İkinci sayı : ");
    scanf("%d", &b);

    int toplam = a + b;
    printf("%d + %d = %d\n", a, b, toplam);
```

ÇIKTI

```
$ ./toplam
Birinci sayı: 12
İkinci sayı : 30
12 + 30 = 42
```

Yeni öge: `scanf("%d", &a)` bir tamsayı okuyup `a`'nın adresine yazar; istemlerin sonunda `\n` olmadığından imleç aynı satırda bekler. **Değiştir:** (a) `&a`'daki `&`'ı silip derleyin ve uyarıyı okuyun (HA1). (b) Üç sayının toplamını bulun.

4.3 Örnek 3 ★ Dikdörtgenin alanı ve çevresi

Problem. Eni ve boyu metre cinsinden tamsayı olan bir arsanın alanını (A = en × boy) ve çevresini (Ç = 2 × (en + boy)) hesaplayalım. Okuma kısmı Örnek 2 ile aynıdır; aşağıda yeni satırlar görülüyor.

H03_03_dikdortgen.c · 18–22. satırlar

```c
    int alan = en * boy;          /* A = en × boy       */
    int cevre = 2 * (en + boy);   /* Ç = 2 × (en + boy) */

    printf("Alan  : %d m²\n", alan);
    printf("Çevre : %d m\n", cevre);
```

ÇIKTI

```
$ ./dikdortgen
Dikdörtgenin eni (m) : 7
Dikdörtgenin boyu (m): 4
Alan  : 28 m²
Çevre : 22 m
```

Yeni öge: formül operatörlerle birebir koda aktarılır; parantezler matematikteki anlamıyla kullanılır (öncelik H4'te). **Değiştir:** Kenarlar 7,5 m gibi kesirli olsaydı hangi tipi ve belirteçleri kullanırdınız? (Bir sonraki örnek.)

4.4 Örnek 4 ★ Dairenin alanı: double, #define, const ve %.2f

Problem. Yarıçapı cm cinsinden verilen dairenin alanını cm² ve m² olarak yazdıralım.

H03_04_daire_alani.c (6. satırdan; başlık yorumu hariç)

```c
#include <stdio.h>
#define PI 3.14159265358979        /* makro: önişlemci metni yerine koyar */

int main(void)
{
    const double CM2_M2 = 0.0001;  /* const: tipli, değiştirilemez sabit */
    double yaricap;

    printf("Yarıçap (cm): ");
    scanf("%lf", &yaricap);        /* double okumak için %lf */

    double alan = PI * yaricap * yaricap;
    printf("Alan = %f cm² (varsayılan 6 basamak)\n", alan);
    printf("Alan = %.2f cm²\n", alan);
    printf("Alan = %.4f m²\n", alan * CM2_M2);
    return 0;
}
```

ÇIKTI

```
$ ./daire
Yarıçap (cm): 5
Alan = 78.539816 cm² (varsayılan 6 basamak)
Alan = 78.54 cm²
Alan = 0.0079 m²
```

Yeni ögeler: `#define PI` önişlemci sabiti, `const double CM2_M2` tipli salt okunur sabit; double okumak için `%lf`, yazmak için `%f` (varsayılan 6 basamak) ve `%.2f`. **Değiştir:** (a) main içine `CM2_M2 = 1.0;` ekleyin: "assignment of read-only variable 'CM2_M2'" hatası. (b) Çevreyi (2πr) de hesaplayın.

4.5 Örnek 5 ★★ Celsius → Fahrenheit (amiral örnek)

1. Problem. Bir hava durumu uygulaması, kullanıcının girdiği Celsius sıcaklığını Fahrenheit'a çevirecek: F = C × 9/5 + 32.

2. Çözümleme. Girdi kesirli (36.6) ve negatif olabilir → `double`. Denetim noktaları: 0 °C = 32 °F, 100 °C = 212 °F ve −40 °C = −40 °F (iki ölçeğin kesiştiği tek nokta). Kullanıcı sayı yerine harf girebilir.

Tablo 3.6
Örnek 5'in IPO çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• `celsius`: kesirli, negatif olabilir	• `f = celsius × 9 / 5 + 32` • Bölme **kayan noktalı** olmalı	• F değeri, 1 ondalık basamakla

3. Algoritma. `OKU celsius` → `f ← celsius × 9 / 5 + 32` → `YAZ f`. **4. C kodu:**

H03_05_celsius_fahrenheit.c · 6–21. satırlar (1–5: başlık yorumu)

```
 6  #include <stdio.h>
 7
 8  int main(void)
 9  {
10      double celsius = 0.0;          /* okuma başarısız olursa 0 kalır */
11
12      printf("Sıcaklık (°C): ");
13      scanf("%lf", &celsius);
14
15      double f_hatali = celsius * (9 / 5) + 32;     /* 9 / 5 → 1 (!) */
16      double f_dogru = celsius * 9.0 / 5.0 + 32;    /* 9.0 / 5.0 → 1.8 */
17
18      printf("Hatalı formül: %.1f °F\n", f_hatali);
19      printf("Doğru formül : %.1f °F\n", f_dogru);
20      return 0;
21  }
```

5. Satır satır açıklama.

- **10. satır:** Okuma başarısız olursa değişken değişmez; tanımsız bir değer yerine bilinen 0.0 ile başlatıyoruz. **13. satır:** double için `%lf`.
- **15. satır — tuzak:** `9 / 5` ifadesinde **iki işlenen de tamsayıdır**; bölme tamsayı bölmesidir ve kesirli kısım atılır: 9 / 5 = 1. Formül sessizce F = C + 32'ye dönüşür. Derleyici uyarmaz; bu geçerli bir C'dir.
- **16. satır:** `9.0 / 5.0` kayan noktalı bölmedir: 1.8. İşlenenlerden birinin double olması yeterlidir (dönüşüm kuralları H4'te). **18–19. satırlar:** `%.1f` ile bir ondalık basamak.

6. İzleme tablosu (girdi 36.6):

Tablo 3.7
Örnek 5'in izleme tablosu

Adım	Satır	celsius	f_hatali	f_dogru	Çıktı
1	10	0.0	–	–	
2	13	**36.6**	–	–	(girdi okunur)
3	15	36.6	**68.6**	–	
4	16	36.6	68.6	**97.88**	
5	18	36.6	68.6	97.88	**Hatalı formül: 68.6 °F**
6	19	36.6	68.6	97.88	**Doğru formül : 97.9 °F**

7. Örnek çalıştırma:

```
ÇIKTI
$ ./celsius
Sıcaklık (°C): 36.6
Hatalı formül: 68.6 °F
Doğru formül : 97.9 °F
```

8. Test tablosu (gerçek çalıştırmalar):

Tablo 3.8
Örnek 5'in test tablosu

Girdi (tür)	Beklenen	Gerçek: hatalı / doğru	Sonuç
`36.6` (normal)	97.9 °F	68.6 / 97.9	Doğru formül geçti
`100` (kaynama noktası)	212.0 °F	132.0 / 212.0	Doğru formül geçti
`-40` (iki ölçeğin kesiştiği nokta)	-40.0 °F	-8.0 / -40.0	Doğru formül geçti
`0` (donma noktası)	32.0 °F	32.0 / 32.0	Doğru formül geçti
`otuz` (hatalı girdi)	hata bildirilmeli	32.0 / 32.0	Uyarı yok: H5–H6'da doğrulama

Ders: **yalnızca 0 ile test etseydik hatayı yakalayamazdık**; 0 girdisinde iki formül de 32.0 verir. Sınır değerlerin yanında sıradan ve "ilginç" değerler (−40) de denenmelidir. Hatalı girdide program sessizce 32.0 °F yazıyor; bunu H5–H6'da girdi doğrulamayla düzelteceğiz.

9. Değiştir – genişlet. (a) Tersini yazın: C = (F − 32) × 5 / 9. (b) Kelvin'i ekleyin: K = C + 273.15. (c) `celsius * 9 / 5` yazsaydık sonuç doğru olur muydu? (İpucu: soldan sağa; H4.)

4.6 Örnek 6 ★★ Makinenizde veri tipi boyutları tablosu

Problem. Makinedeki temel tiplerin boyutlarını ve aralıklarını listeleyelim. Programın gerçek çıktısı Tablo 3.2'de verilmiş ve Bölüm 3.4'te yorumlanmıştır; aşağıda programın ilk satırları görülüyor (tamamı dosyada).

H03_06_tip_boyutlari.c · 17–22. satırlar

```
    printf("int           %4zu  %20d  %20d\n",
           sizeof(int), INT_MIN, INT_MAX);
    printf("unsigned int  %4zu  %20u  %20u\n",
           sizeof(unsigned int), 0u, UINT_MAX);
    printf("long          %4zu  %20ld  %20ld\n",
           sizeof(long), LONG_MIN, LONG_MAX);
```

Yeni ögeler: `sizeof`, `%zu`, `<limits.h>`/`<float.h>` sabitleri. Uzun `printf` çağrıları iki satıra bölünmüştür; bir deyim noktalı virgüle kadar sürer. **Keşif (Atölye 3):** Programı Windows'ta çalıştırın; hangi satır değişir, neden?

4.7 Örnek 7 ★★ Karakter ↔ ASCII kodu dönüşümü

H03_07_karakter_ascii.c · 10–18. satırlar

```
    char harf = 'A';
    char sonraki = harf + 1;       /* 65 + 1 = 66 → 'B' */

    printf("'%c' karakterinin kodu: %d\n", harf, harf);
    printf("'%c' + 1 = '%c' (kod %d)\n", harf, sonraki, sonraki);
    printf("'a' = %d, 'A' = %d, fark = %d\n", 'a', 'A', 'a' - 'A');
    printf("'7' karakterinin kodu %d, 7 sayısı ise %d\n", '7', 7);
    printf("'7' - '0' = %d (rakam karakterinden sayıya)\n", '7' - '0');
    printf("'g' büyük harfe: '%c'\n", 'g' - ('a' - 'A'));
```

ÇIKTI

```
$ ./ascii
'A' karakterinin kodu: 65
'A' + 1 = 'B' (kod 66)
'a' = 97, 'A' = 65, fark = 32
'7' karakterinin kodu 55, 7 sayısı ise 7
'7' - '0' = 7 (rakam karakterinden sayıya)
'g' büyük harfe: 'G'
sizeof(char) = 1, sizeof('A') = 4
```

Yeni öge: aynı değer `%c` ile karakter, `%d` ile sayı olarak yazılır. `'7' - '0'` deseni rakam karakterini sayıya çevirir; H13'te kendi `atoi`'mizi bu fikirle yazacağız. **Değiştir:** `'Z' + 1` hangi karakteri verir? Önce Şekil 3.6'ya bakarak tahmin edin.

4.8 Örnek 8 ★★ İki değişkenin değerlerini takas etme

Problem. `a` ile `b`'nin değerlerini değiş tokuş edelim. `a = b; b = a;` neden çalışmaz? Çünkü ilk atama a'nın eski değerini siler; iki bardaktaki sıvıyı değiştirmek için üçüncü, boş bir bardak gerekir.

H03_08_takas.c (6. satırdan; başlık yorumu hariç)

```c
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 9;
    printf("Önce : a = %d, b = %d\n", a, b);

    int gecici = a;    /* 1. adım: a'yı boş bardağa dök */
    a = b;             /* 2. adım: b'yi a'ya dök        */
    b = gecici;        /* 3. adım: boş bardaktakini b'ye dök */

    printf("Sonra: a = %d, b = %d\n", a, b);
    return 0;
}
```

Tablo 3.9
Örnek 8'in izleme tablosu (Çıktı sütunu gerçek çıktıyla aynıdır)

Adım	Deyim	a	b	gecici	Çıktı
1	`int a = 5; int b = 9;`	5	9	–	
2	`printf("Önce…")`	5	9	–	`Önce : a = 5, b = 9`
3	`int gecici = a;`	5	9	**5**	
4	`a = b;`	**9**	9	5	
5	`b = gecici;`	9	**5**	5	
6	`printf("Sonra…")`	9	5	5	`Sonra: a = 9, b = 5`

Takas, AP-2'deki sıralama algoritmalarının temel işlemidir; H8'de fonksiyona, H12'de işaretçili fonksiyona dönüşecek. **Değiştir (★★★):** `a = a + b; b = a - b; a = a - b;` geçici değişkensiz takas yapar. Hangi değerlerde tehlikelidir? (İpucu: Bölüm 3.5.)

4.9 Örnek 9 ★★ Değişkenlerin adreslerini yazdırma

H03_09_adresler.c · 10–18. satırlar

```c
    int x = 42;
    int y = 7;
    double d = 3.5;
    char c = 'Z';

    printf("x: değer %4d   adres %p   boyut %zu\n", x, (void *)&x, sizeof(x));
    printf("y: değer %4d   adres %p   boyut %zu\n", y, (void *)&y, sizeof(y));
    printf("d: değer %4.1f   adres %p   boyut %zu\n", d, (void *)&d, sizeof(d));
    printf("c: değer %4c   adres %p   boyut %zu\n", c, (void *)&c, sizeof(c));
```

ÇIKTI · 1. ÇALIŞTIRMA

```
$ ./adresler
x: değer   42   adres 0x7ffcf96cd208   boyut 4
y: değer    7   adres 0x7ffcf96cd20c   boyut 4
d: değer  3.5   adres 0x7ffcf96cd210   boyut 8
c: değer    Z   adres 0x7ffcf96cd207   boyut 1
```

Yeni ögeler: `&x`, x'in adresidir; `%p` adresi onaltılı yazar ve bir `void *` beklediği için adresi `(void *)` ile dönüştürürüz (H12). **Adresler her çalıştırmada farklıdır** (ikinci çalıştırmada x'in adresi `0x7ffdfc93ce28` idi): işletim sistemi güvenlik için programın bellekteki yerleşimini her çalıştırmada rastgele kaydırır (ASLR, address space layout randomization). Değişmeyen düzendir: x ile y arasında 4 bayt, y ile d arasında 4 bayt; c, x'ten hemen önceki baytta. Bu sıra derleyiciye bağlıdır; standart bir yerleşim sırası garanti etmez. Programı gdb hata ayıklayıcısında durdurup baytları okuduk (gdb ASLR'yi kapattığı için adresler sabittir): `x = 42` → `2a 00 00 00`, `d = 3.5` → `00 00 00 00 00 00 0c 40` (ham döküm: `kodlar/ciktilar/H03_09_adresler_gdb.txt`; bayt şeridi sunumda ve web sayfasında).

4.10 Örnek 10 ★★ Vücut kitle indeksi hesaplayıcı (amiral örnek)

1. Problem. Kilo (kg) ve boy (cm) alıp vücut kitle indeksini (VKİ = kilo / boy², boy metre cinsinden) hesaplayan ve hizalı bir rapor yazdıran bir program yazalım. Rapor, Dünya Sağlık Örgütünün yetişkinler için kullandığı sınıf aralıklarını başvuru olarak göstersin (World Health Organization [WHO], 2000); sınıfı otomatik belirlemek karar yapısı gerektirir (H5).

2. Çözümleme. Kilo ve boy pozitif ve kesirli olabilir → `double`. Boy 0 girilirse sıfıra bölme olur; double'da bu bir çökme değil, IEEE 754'ün özel değeri **inf** (sonsuz) üretir. Sayı olmayan girdi okunamaz. Birim karışıklığı (boyu metre girmek) sonucu 10 000 kat büyütür; istemler birimi açıkça söylemelidir.

Tablo 3.10
Örnek 10'un IPO çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• `kilo` (kg) • `boy_cm` (cm)	• `boy_m = boy_cm / 100.0` • `vki = kilo / (boy_m × boy_m)`	• Hizalı rapor: kilo, boy, VKİ • Başvuru aralıkları

3. Algoritma. `OKU kilo, boy_cm` → `boy_m ← boy_cm / 100` → `vki ← kilo / (boy_m × boy_m)` → `YAZ rapor`. **4. C kodu:**

H03_10_vki.c · 6–30. satırlar (1–5: başlık yorumu)

```
 6  #include <stdio.h>
 7
 8  int main(void)
 9  {
10      double kilo = 0.0;
11      double boy_cm = 0.0;
12
13      printf("Kilonuz (kg): ");
14      scanf("%lf", &kilo);
15      printf("Boyunuz (cm): ");
16      scanf("%lf", &boy_cm);
17
```

```c
    double boy_m = boy_cm / 100.0;          /* cm → m */
    double vki = kilo / (boy_m * boy_m);    /* VKİ = kg / m² */

    printf("\n========== VKİ RAPORU ==========\n");
    printf("Kilo            : %7.1f kg\n", kilo);
    printf("Boy             : %7.2f m\n", boy_m);
    printf("Vücut kitle ind. : %7.2f kg/m²\n", vki);
    printf("---------------------------------\n");
    printf("Başvuru (DSÖ, yetişkin):\n");
    printf("  18,5 altı zayıf  | 18,5–24,9 normal\n");
    printf("  25–29,9 fazla kilolu | 30 ve üstü obez\n");
    return 0;
}
```

5. Satır satır açıklama.

- **10–11. satırlar:** Okuma başarısız olursa bilinen bir değerde kalsınlar diye 0.0 ile başlatılır. **18. satır:** `100.0` ile bölmek bölmenin kayan noktalı olmasını garanti eder (Örnek 5'in dersi).
- **19. satır:** parantez şart: `kilo / boy_m * boy_m` önce böler, sonra çarpar (sonuç = kilo).
- **22–24. satırlar:** `%7.1f`, `%7.2f` sayıları 7 karakterlik alanda sağa yaslar.

6. İzleme tablosu (girdi 70, 175):

Tablo 3.11
Örnek 10'un izleme tablosu

Adım	Satır	kilo	boy_cm	boy_m	vki
1	10–11	0.0	0.0	–	–
2	14	**70.0**	0.0	–	–
3	16	70.0	**175.0**	–	–
4	18	70.0	175.0	**1.75**	–
5	19	70.0	175.0	1.75	**22.857…**

7. Örnek çalıştırma:

ÇIKTI

```
$ ./vki
Kilonuz (kg): 70
Boyunuz (cm): 175

========== VKİ RAPORU ==========
Kilo            :    70.0 kg
Boy             :    1.75 m
Vücut kitle ind. :   22.86 kg/m²
---------------------------------
Başvuru (DSÖ, yetişkin):
  18,5 altı zayıf  | 18,5–24,9 normal
  25–29,9 fazla kilolu | 30 ve üstü obez
```

8. Test tablosu (gerçek çalıştırmalar):

Tablo 3.12
Örnek 10'un test tablosu

Girdi (tür)	Beklenen VKİ	Gerçek VKİ	Sonuç
70, 175 (normal)	22.86	22.86	Geçti
45.5, 160 (ondalıklı girdi)	17.77	17.77	Geçti
70, 0 (sınır: boy 0)	hata bildirilmeli	inf	Yakalanmadı: H5'te doğrulama
yetmiş (hatalı girdi)	hata bildirilmeli	-nan	Yakalanmadı: H5'te doğrulama

`inf`, 70'in 0'a bölünmesinin IEEE 754'teki sonucudur. Hatalı girdide kilo ve boy 0 kalır ve 0/0 **NaN** (not a number) üretir; glibc bunu `-nan` yazar. Program çökmez ama anlamsız bir rapor üretir: girdi doğrulamanın (H5) somut gerekçesi. **9. Değiştir – genişlet.** (a) Boyu metre cinsinden okuyan sürümü yazın. (b) VKİ 18,5 ve 24,9 için ideal kilo aralığını (kilo = VKİ × boy_m²) rapora ekleyin. (c) `%7.2f` yerine `%10.3f` kullanın; hizalama nasıl değişir?

4.11 Örnek 11 ★★★ Tamsayı taşması deneyi

Problem. İşaretsiz ve işaretli tamsayılar en büyük değerlerini aşınca ne olduğunu deneyelim ve C standardının iki durumu neden farklı ele aldığını gözlemleyelim.

H03_11_tasma_deneyi.c · 6–25. satırlar (1–5: başlık yorumu)

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    unsigned int sayac = UINT_MAX;          /* 2^32 - 1 */
    printf("unsigned int en büyük : %u\n", sayac);
    sayac = sayac + 1;                      /* TANIMLI: mod 2^32 → 0 */
    printf("bir fazlası           : %u\n", sayac);

    int adim = 0;
    printf("INT_MAX'a kaç ekleyelim? ");
    scanf("%d", &adim);                     /* değer çalışma zamanında */

    int x = INT_MAX;                        /* 2^31 - 1 */
    printf("int en büyük          : %d\n", x);
    x = x + adim;                           /* taşarsa TANIMSIZ DAVRANIŞ */
    printf("INT_MAX + %d           : %d\n", adim, x);
    return 0;
}
```

13. satır: işaretsiz toplama 2^{32} modunda **tanımlıdır** (4294967295 + 1 = 0). **18. satır:** eklenecek değeri kullanıcıdan okumak taşmayı çalışma zamanına bırakır (`INT_MAX + 1` sabit ifadesi `-Woverflow` uyarısı verirdi). **22. satır:** adim > 0 ise işaretli taşma olur: **tanımsız davranış**; bu makinede −2147483648 çıktı, ama bu bir güvence değildir.

Normal derlemede adim = 1 için son satır `INT_MAX + 1 : -2147483648` olur; adim = 0 ve −1 ile taşma yoktur (`2147483647`, `2147483646`; tüm çalıştırmalar `kodlar/ciktilar/` altında). GCC'nin **tanımsız davranış denetleyicisiyle** (UndefinedBehaviorSanitizer, `-fsanitize=undefined`) derlendiğinde ise taşma anında çalışma zamanı uyarısı alınır; denetleyici hatayı raporlar ve programın sürmesine izin verir:

ÇIKTI · TANIMSIZ DAVRANIŞ DENETLEYİCİSİYLE (UBSAN)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -fsanitize=undefined H03_11_tasma_deneyi.c
-o tasma_ub
$ ./tasma_ub
unsigned int en büyük : 4294967295
bir fazlası           : 0
INT_MAX'a kaç ekleyelim? 1
int en büyük          : 2147483647
H03_11_tasma_deneyi.c:22:7: runtime error: signed integer overflow: 2147483647 + 1
cannot be represented in type 'int'
INT_MAX + 1           : -2147483648
```

Değiştir: Aynı deneyi `short` ile yapın: `SHRT_MAX` değerindeki bir `short`'a 1 eklenince toplam önce `int`'e yükseltilir ve taşmaz (32768); sonucu yeniden `short`'a atamak ise derleyiciye bağlı bir dönüşümdür. Farkı yorumlayın.

4.12 Örnek 12 ★★★ 0.1 + 0.2 deneyi ve %.20f

Problem. Haftanın Büyük Sorusunu doğrudan deneyelim: 0.1 + 0.2 gerçekten 0.3 mü?

H03_12_kayan_nokta.c · 10–23. satırlar

```c
    double a = 0.1;
    double b = 0.2;
    double toplam = a + b;

    printf("toplam (%%f)      : %f\n", toplam);
    printf("toplam (%%.20f)   : %.20f\n", toplam);
    printf("0.3 (%%.20f)      : %.20f\n", 0.3);
    printf("0.1 (%%.20f)      : %.20f\n", a);
    printf("toplam - 0.3     : %e\n", toplam - 0.3);
    printf("0.3'e eşit mi?   : %d (1 evet, 0 hayır)\n", toplam == 0.3);
    printf("0.1 (%%a)         : %a\n", a);

    float f = 0.1f;
    printf("float 0.1 (%%.20f): %.20f\n", f);
```

ÇIKTI

```
$ ./kayan
toplam (%f)      : 0.300000
toplam (%.20f)   : 0.30000000000000004441
0.3 (%.20f)      : 0.29999999999999998890
0.1 (%.20f)      : 0.10000000000000000555
toplam - 0.3     : 5.551115e-17
0.3'e eşit mi?   : 0 (1 evet, 0 hayır)
0.1 (%a)         : 0x1.999999999999ap-4
float 0.1 (%.20f): 0.10000000149011611938
```

`%f` varsayılan 6 basamakla 0.300000 yazar ve farkı **gizler**; `%.20f` saklanan değerin gerçek açılımını gösterir. `toplam - 0.3` tam bir double adımıdır (≈ 5.55e-17); karşılaştırma bu yüzden 0 verir. `%a` double'ı onaltılı bilimsel gösterimle yazar: `0x1.999999999999ap-4` = 1,6 × 2^{-4}; Şekil 3.5'teki mantis (`999…9a`) ve üs (−4) doğrudan okunur. float ile 0.1'in hatası çok daha büyüktür. **Değiştir:** `a = 0.5, b = 0.25` yapın; toplam neden bu kez **tam** 0.75 çıkar?

5. Hata Avcısı

Hataların hepsi bu ortamda gerçekten derlenip çalıştırıldı; iletiler GCC 13'ün birebir çıktısıdır (`kodlar/hata_avcisi/`, hatalı ve düzeltilmiş sürümler). Dikkat: HA4 ve HA5 için derleyici **hiç uyarı vermez**; mantık hatalarını ancak test ederek yakalarız.

Tablo 3.13
Hata–ileti–neden–çözüm tablosu

Hata	İleti (özet)	Neden	Çözüm
HA1: scanf'te & yok	`expects argument of type 'int *'` + Segmentation fault	Değer adres sanılıp oraya yazılır	`scanf("%d", &yas);`
HA2: belirteç–tip uyuşmazlığı	`'%d' expects … 'int', but … has type 'double'`	Argüman yanlış yerden okunur (tanımsız)	double → `%f`, int → `%d`
HA3: ilk değer yok	`'toplam' is used uninitialized [-Wuninitialized]`	Yerel değişkenin değeri belirsizdir	`int toplam = 0;`
HA4: 5 / 2 ve ==	(uyarı yok)	Kesirli kısım atılır; kayan nokta tam değil	`/ 2.0`; farka bak
HA5: tampondaki '\n'	(uyarı yok) istem atlanır	`%c` beyaz karakter atlamaz	`scanf(" %c", &c);` (3.10, Ek örnek 15)

HA1 · scanf'te & operatörünün unutulması

H03_HA1_scanf_ampersand.c · 8–12. satırlardan

```c
    int yas = 0;
    printf("Yaşınız: ");
    scanf("%d", yas);            /* HATA: & unutuldu */
    printf("Yaş: %d\n", yas);
```

DERLEYİCİ İLETİSİ (UYARI SATIRLARI) VE ÇALIŞTIRMA

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H03_HA1_scanf_ampersand.c -o ha1
H03_HA1_scanf_ampersand.c:11:13: warning: format '%d' expects argument of type 'int
*', but argument 2 has type 'int' [-Wformat=]
$ ./ha1
Yaşınız: 25
bash: line 1: Segmentation fault      ./ha1
[çıkış kodu: 139]
```

Derleyici `%d` için `int *` (bir int'in adresi) beklediğini, `int` verildiğini söylüyor. Program çalışınca scanf, `yas`'ın değeri olan 0'ı adres sanıp oraya yazmaya çalışır; işletim sistemi bu erişimi durdurur (Segmentation fault; çıkış kodu 139 = 128 + 11, SIGSEGV). **Düzeltilmiş sürüm** (`H03_HA1_scanf_ampersand_duzeltilmis.c`) uyarısız derlenir ve `Yaş: 25` yazar.

HA2 · Biçim belirteci–tip uyuşmazlığı

DERLEYİCİ İLETİSİ (UYARI SATIRLARI) VE ÇALIŞTIRMA

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H03_HA2_bicim_uyumsuzlugu.c -o ha2
H03_HA2_bicim_uyumsuzlugu.c:11:26: warning: format ‘%d’ expects argument of type
‘int’, but argument 2 has type ‘double’ [-Wformat=]
H03_HA2_bicim_uyumsuzlugu.c:12:26: warning: format ‘%f’ expects argument of type
‘double’, but argument 2 has type ‘int’ [-Wformat=]
$ ./ha2
Ortalama  : 1840679640
Not sayısı: 77.500000
[çıkış kodu: 0]
```

Kaputun altı: x86-64 Linux'ta int argümanlar genel amaçlı yazmaçlarda, double argümanlar ayrı kayan nokta (XMM) yazmaçlarında geçirilir. `%d` tamsayı yazmacına baktığı için orada duran rastgele 1840679640 sayısını yazdı; ikinci satırda `%f` kayan nokta yazmacında önceki çağrıdan kalan 77.5'i buldu. Sonuçlar rastlantısaldır: tanımsız davranış. **Düzeltilmiş sürüm** (`H03_HA2_bicim_uyumsuzlugu_duzeltilmis.c`) uyarısız derlenir ve `Not sayısı: 2` yazar.

HA3 · İlk değer atanmamış değişken (`H03_HA3_ilk_deger_yok.c`). `int toplam;` bildirilip `toplam = toplam + sayi;` yazıldığında GCC "'toplam' is used uninitialized [-Wuninitialized]" uyarısını verdi; program 10 yerine 32775 yazdı. `-Wall` bu hatayı sizin yerinize buldu: uyarıları asla görmezden gelmeyin. Düzeltme: `int toplam = 0;`.

HA4 · Tamsayı bölmesi ve kayan noktalı eşitlik (`H03_HA4_bolme_ve_esitlik.c`). `double ortalama = (a + b) / 2;` satırında iki işlenen de int olduğundan 7 / 2 = 3 olur; sonuç double bir değişkene atansa bile kesirli kısım çoktan atılmıştır (program `ortalama = 3.000000` yazdı). `0.1 * 3 == 0.3` ise 0 verdi. Derleyici ikisini de geçerli C sayar ve **uyarmaz**. Düzeltilmiş sürüm `/ 2.0` kullanır (`ortalama = 3.500000`) ve eşitlik yerine farka bakar.

6. Mühendis Gözüyle

6.1 Tip seçimi bir tasarım kararıdır

Tip seçmek, değişkenin hangi değerleri alabileceğine dair bir sözleşmedir. Basit karar yolu: değer kesirli olabiliyorsa `double`; tek karakterse `char`; ±2,1 milyarı aşabiliyorsa `long long`; aksi hâlde `int` (sunumdaki akış diyagramı).

- **Aralığı önceden hesaplayın.** Dünya nüfusu (yaklaşık 8 milyar) `int`'e sığmaz. Otuz gün milisaniye cinsinden 30 × 24 × 3600 × 1000 = 2 592 000 000'dur ve `INT_MAX`'ı aşar: 32 bitlik işaretli bir milisaniye sayacı yaklaşık 24,8 günde dolar.
- **Kimlikler sayı değildir.** Öğrenci ya da T.C. kimlik numarası üzerinde aritmetik yapılmaz; öğrenci numarası gibi kodların başında sıfır da olabilir (sayı olarak saklanınca kaybolur). Bu hafta Mini-OBS'de `long long`, H13'te karakter dizisi kullanacağız.
- **Boyut varsaymayın, ölçün; derleyiciyi yardıma çağırın.** `long` Linux'ta 8, Windows'ta 4 bayttır; `<stdint.h>`'deki `int32_t`, `int64_t` her platformda aynı boyuttadır (H14). `-Wall -Wextra -pedantic` HA1–HA3'ü, `-fsanitize=undefined` işaretli taşmayı yakaladı; `-O2` ile derlemede glibc, dönüş değeri kullanılmayan scanf'ler için `-Wunused-result` uyarısı da verdi (H6). Sınır değerleri test edin: Örnek 5'te 0 girdisi hatayı gizledi.

6.2 Birikimli hata: Patriot hesabını yeniden yapmak

GAO raporu, farklı çalışma sürelerinde saat hatasının ne kadar büyüdüğünü tablolaştırır. 0,1'in ikili açılımı ikili noktadan sonra 23 bitte kesilirse (838860 / 2^{23}) her 0,1 saniyelik tıkta yaklaşık $9{,}5 \times 10^{-8}$ saniyelik hata oluşur. Ek örnek 16 (`const double KESILMIS = 838860.0 / 8388608.0;`) bu hatayı tık sayısıyla çarpar. Programın gerçek çıktısı: 1 saat → 0.0034 s, 8 saat → 0.0275 s, 20 saat → 0.0687 s, 48 saat → 0.1648 s, 72 saat → 0.2472 s, 100 saat → **0.3433 s** (`kodlar/ciktilar/H03_16_patriot_hesabi.txt`). Bu değerler GAO tablosunun hata sütunuyla aynıdır; yalnızca raporda 8 saat satırında .0025 basılıdır: bu bir dizgi hatasıdır, aynı satırdaki hesaplanan zaman (28799.9725 s) .0275 s hata verir. Aynı tabloya göre 100 saatte radarın izleme penceresi (menzil kapısı, range gate) yaklaşık 687 m kaymıştı (GAO, 1992).

Ders: tek işlemdeki 10^{-8}'lik hata zararsızdır, milyonlarca tekrar sistemin davranışını değiştirir; zaman tamsayı tıklarla tutulmalı, dönüşümler yeterli hassasiyetle ve az sayıda yapılmalıdır.

7. Mini-OBS Projesi: v0.3

Mini-OBS (Öğrenci Not Yönetim Sistemi) her hafta o haftanın kavramlarıyla büyüyen dönem projemizdir. H1'de karşılama ekranını, H2'de IPO tablosunu ve akış diyagramını hazırladık. **v0.3 ilk gerçek veriyi işleyen sürümdür:** tek öğrencinin numarasını, vize ve final notlarını okur, basit ortalamayı hesaplar ve biçimli bir rapor yazdırır.

İpuçları. 2301010045 gibi 10 haneli bir numara `INT_MAX`'ı aşar: Tablo 3.2'ye bakarak tip seçin (`%lld`). Notlar 72.5 gibi olabilir → double (`%lf` ile oku, `%6.2f` ile yaz); ortalamada `/ 2` değil `/ 2.0`. Etiketleri aynı uzunlukta yazıp sayıları sabit genişlikle sağa yaslayın; tüm değişkenleri 0 ile başlatın.

Kabul ölçütleri:

- ☐ `gcc -std=c17 -Wall -Wextra -pedantic` ile sıfır uyarı.
- ☐ 10 haneli öğrenci numarasını doğru okur ve aynen yazar.
- ☐ Kesirli notları kabul eder; ortalamayı iki ondalık basamakla, hizalı yazar.
- ☐ Tüm değişkenlerin ilk değeri vardır; double notlar scanf'te `%lf` ile okunur (printf'te `%f`; `%lf` de geçerlidir).

Tablo 3.14
Mini-OBS v0.3 test senaryoları (referans çözümle gerçek çalıştırma)

Test	Girdi (no, vize, final)	Beklenen ortalama	Referans çözümün çıktısı
T1 normal	2301010045, 70, 85	77.50	77.50
T2 kesirli not	2301010045, 72.5, 88	80.25	80.25
T3 sınır: 0 ve 100	2301010045, 0, 100	50.00	50.00
T4 sınır: en yüksek	2301010045, 100, 100	100.00	100.00
T5 hatalı girdi (sayı değil)	abc	hata bildirilmeli (v0.5)	0.00
T6 aralık dışı notlar	2301010045, 120, -5	reddedilmeli (v0.5)	57.50

Tam ekran çıktıları: `91_Proje_MiniOBS/v0.3/mini_obs_v0_3_ciktilar.txt`. T5 ve T6 v0.3'ün bilinçli sınırlarıdır (sayı olmayan girdi, 0–100 dışı not); ağırlıklı ortalama v0.4'te (H4), girdi doğrulama v0.5'te (H5) gelecek.

8. Alıştırmalar ve Atölye

8.1 Atölye etkinlikleri

- **Atölye 1 · Kişisel bilgi kartı.** Doğum yılı, boy, kan grubu (tek harf) ve öğrenci numarasını çerçeveli bir kart olarak yazdırın; sağ kenarı `%-17d` gibi sola yaslı alanlarla hizalayın (Türkçe harfler UTF-8'de 2 bayt olduğundan metin genişlikleri kayar, sayılar kaymaz).
- **Atölye 2 · Birim dönüştürücü.** Uzunluk (1 inç = 2,54 cm), sıcaklık (°C → °F ve K) ve süre (saniye → dakika ve saat) dönüşümleri; her katsayı `const`, tamsayı bölmesi tuzağına dikkat.
- **Atölye 3 · Keşif: "Makinemde tipler kaç bayt? Neden arkadaşımınkinden farklı?"** Örnek 6'yı çalıştırıp Tablo 3.2 ile karşılaştırın; farkı LP64/LLP64 ile açıklayın.

8.2 Alıştırmalar

Önce tahmin edin, sonra derleyip kontrol edin (çözümler öğrenci sürümünde yok).

Alıştırma 3.1 ★★☆ *A · Çıktı tahmini*

Her parçanın çıktısını tahmin edin: (a) ★ (b) ★★ (c) ★★★; (c) C'de tanımlı mı?

alistirmalar/H03_A01_cikti_tahmini.c (main gövdesinden)

```c
/* (a) */ int a = 7; int b = 2;
printf("%d %d\n", a / b, b / a);
printf("%.2f\n", a / 2.0);
/* (b) */ printf("[%6.2f][%-4d][%03d]\n", 2.5, 7, 7);
/* (c) */ unsigned char u = 250;
u = u + 10;
printf("%d\n", u);
```

Alıştırma 3.2 ★★☆ *B · Hata bul*

(a) ★ Derlenir ama çalışınca çöker. (b) ★★ Ortalama 72.5 yerine anlamsız bir sayı çıkıyor: iki hatayı bulun. (c) ★★ Yaş girilince cinsiyet sorulmadan geçiliyor: tek karakterlik düzeltmeyi yapın.

alistirmalar/H03_A02_hata_bul.c (main gövdesinden)

```c
/* (a) */ int yas = 0;
scanf("%d", yas);
/* (b) */ int vize = 70; int final = 75;
double ortalama = (vize + final) / 2;
printf("Ortalama: %d\n", ortalama);
/* (c) */ scanf("%d", &yas);
char cinsiyet = '?';
scanf("%c", &cinsiyet);
```

Alıştırma 3.3 ★★☆ *C · Parsons*

Satırları, iki tamsayı okuyup ortalamalarını 2 ondalık basamakla yazan bir `main` gövdesi oluşturacak biçimde sıralayın. Bir satır **çeldiricidir**; onu bulun.

alistirmalar/H03_A03_parsons.c (main gövdesinden)

```
printf("Ortalama: %.2f\n", ortalama);
int a = 0;
double ortalama = (a + b) / 2;
scanf("%d %d", &a, &b);
int b = 0;
double ortalama = (a + b) / 2.0;
printf("İki tamsayı: ");
```

Alıştırma 3.4 ★★☆ *D · Kod tamamla*

Boşlukları doldurun; çıktı sırasıyla `8100000000`, `4000000000`, `A`, `45.7%`, `ff` olmalı.

alistirmalar/H03_A04_belirtec_tamamla.c (main gövdesinden)

```
long long nufus = 8100000000LL;
unsigned int buyuk = 4000000000u;
double oran = 45.678;
printf("Nüfus: ___\n", nufus);
printf("Büyük: ___\n", buyuk);
printf("Harf : ___\n", 'A');
printf("Oran : _____\n", oran);
printf("Onaltılı: ___\n", 255);
```

Alıştırma 3.5 ★☆☆ *E · Kod yaz*

Kenarı cm cinsinden (kesirli olabilir) okunan bir küpün hacmini ve yüzey alanını iki ondalık basamakla yazdırın. Deneme: kenar 3 → hacim 27.00, yüzey 54.00.

Alıştırma 3.6 ★★☆ *E · Kod yaz*

Küçük bir harf okuyun; ASCII kodunu, bir sonraki karakteri ve büyük harfini yazdırın. Okumayı tampon tuzağına karşı güvenli yapın.

Alıştırma 3.7 ★★★ *E · Kod yaz*

Zamanı 1970'ten beri saniye olarak **işaretsiz** 32 bitlik sayaçla tutan bir sistem kaç yıl dayanır? `UINT_MAX` saniyeyi yıla çeviren bir program yazın (1 yıl = 365,25 gün); 2038 problemiyle karşılaştırın.

9. Kendini Değerlendirme

- ☐ Bir programın parçalarını adlandırabiliyor, anlamlı tanımlayıcılar seçip bir değişkeni ad, tip, değer ve adresle bellek kutusu olarak çizebiliyorum (3.1–3.3).
- ☐ Tip boyutlarını `sizeof` ile ölçüp LP64/LLP64 farkını açıklayabiliyor; 8 bitlik ikiye tümleyeni bulup işaretsiz sarma ile işaretli taşma (tanımsız) farkını anlatabiliyorum (3.4–3.5).
- ☐ 0.1 + 0.2'nin neden 0.3 etmediğini ve `==` tuzağını açıklayabiliyorum (3.6).
- ☐ `printf`/`scanf` belirteçlerini, `&`'yi, dönüş değerini ve `" %c"`'yi kullanabiliyorum (3.9–3.10).

10. AP-2 Köprüsü ve Merak Kutusu

> AP-2 KÖPRÜSÜ · **Taşma ve algoritma doğruluğu**
>
> - Tip seçimi **doğruluğu** etkiler: doğru bir algoritma, bir ara sonuç taşarsa yanlış sonuç verir.
> - Klasik örnek: iki büyük indisin ortalaması `(alt + ust) / 2` hesaplanırken toplam `INT_MAX`'ı aşabilir; `alt + (ust - alt) / 2` aynı değeri taşmadan verir (AP-2'de arama algoritmalarında).

> MERAK KUTUSU · **Derleyici işaretli taşmanın hiç olmayacağını varsayar**
>
> Ek örnek 17'de normal derlemede toplam −2147483648 çıkarken `x + adim > x` karşılaştırması yine de **1 (doğru)** der! Taşma tanımsız olduğundan GCC ifadeyi `adim > 0`'a sadeleştirir; `-fsanitize=undefined` ile iki taşma raporlanır ve sonuç 0 olur. Aynı kaynak, aynı girdi, iki yanıt: tanımsız davranış budur.

11. Özet, Kavram Haritası ve Terim Sözlüğü

11.1 Haftanın özeti

- Değişken; adı, tipi, değeri ve adresi olan bir bellek bölgesidir (bellek kutusu). Tip, bayt sayısını ve bit yorumunu belirler; boyutlar platforma bağlıdır (`long`: Linux 8, Windows 4 bayt).
- İşaretli tamsayılar ikiye tümleyenle saklanır; işaretsiz taşma sarar, işaretli taşma tanımsızdır. Kayan noktalı sayılar IEEE 754'tür: 0.1 tam saklanamaz, `==` kullanılmaz. `char` bir tamsayıdır.
- `printf` belirteçleri tiple uyuşmalıdır; `scanf` adres (&) ister, okunan öge sayısını döndürür, tamponda kalanı bırakır. Kavram haritası: Şekil 3.8.

11.2 Büyük Sorunun yanıtı

Neden 0.1 + 0.2 ≠ 0.3? 0,1, 0,2 ve 0,3 ikili tabanda sonlu yazılamaz; her biri en yakın double'a yuvarlanır ve 0.1 + 0.2'nin yuvarlanmış toplamı, 0.3'ün yuvarlanmış değerinin bir sonraki komşusuna düşer (fark 2^{-54}). Patriot olayı bu farkın tekrarlarla nasıl büyüdüğünü gösterir.

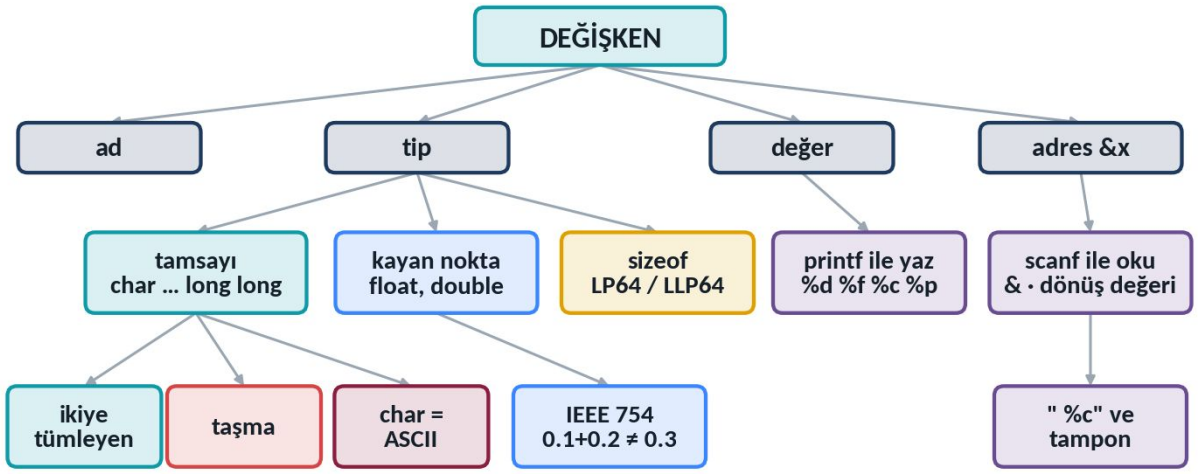
Bir sayaç neden birdenbire eksi değere düşer? 32 bitlik işaretli bir sayaç en fazla 2 147 483 647'yi tutar; ikiye tümleyen gösterimde bir sonraki bit örüntüsü −2 147 483 648'dir. C'de bu taşma tanımsız davranıştır: çoğu makinede eksiye sarar ama buna güvenilemez. Gangnam Style ve 2038 aynı sınırın öyküleridir; çözüm, aralığı yeten bir tip (64 bit) seçmektir.

11.3 Terim sözlüğü (Bölüm 6.8 terim standardıyla uyumlu)

Türkçe	İngilizce	Kısa tanım
değişken; veri tipi	variable; data type	Adlı bellek bölgesi; bayt ve bit yorumu
bildirim; ilk değer; atama	declaration; initialization; assignment	Yer ayırma; ilk değer; sonradan yazma
değişmez; biçim belirteci	literal; format specifier	Koddaki değer; %d gibi dönüşüm
taşma; tanımsız davranış	overflow; undefined behavior	Aralığa sığmama; güvencesiz durum
ikiye tümleyen; kayan nokta; küçük-uçlu	two's complement; floating point; little-endian	Negatif tamsayılar; işaret–üs–mantis; düşük bayt önce

Haftaya hazırlık — H4'e köprü. H4'te operatörlerle (tamsayı bölmesi, %, öncelik, bit operatörleri, cast) çalışacağız; Büyük Soru: tek bir tip dönüşümü bir roketi nasıl düşürebilir? (Ariane 5, 1996.) Hazırlık: Deitel ve Deitel (2022), veri tipleri ve biçimli G/Ç.

Şekil 3.8
Haftanın kavram haritası



12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

GAO. (1992). *Patriot missile defense: Software problem led to system failure at Dhahran, Saudi Arabia* (GAO/IMTEC-92-26). U.S. General Accounting Office.

Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, *23*(1), 5–48. https://doi.org/10.1145/103162.103163

Guinness World Records. (t.y.). *First YouTube video to require a 64-bit counter*. https://www.guinnessworldrecords.com/world-records/381953-first-youtube-video-to-require-a-64-bit-counter

IEEE. (2019). *IEEE standard for floating-point arithmetic* (IEEE Std 754-2019). IEEE.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

World Health Organization. (2000). *Obesity: Preventing and managing the global epidemic* (WHO Technical Report Series No. 894). World Health Organization.